Design Document: AI-Native Compliance Infrastructure

> **Disclaimer:** Independent personal project for self-learning. Not affiliated with any employer, state agency or vendor. Uses only public documents and synthetic data. No internal or confidential data and no personal data (no SSNs, member or patient records, PHI/PII). Views are my own. See DISCLAIMER.md.

Author: Shubham Singh | Status: working prototype, evaluated, known weaknesses documented | Last updated: October 2026

This is the single document covering the project from Phase 0 (setup) to the latest evaluation run: goals, architecture, every phase, key decisions, results, failures, and what is next. Related files: `README.md` (summary), `eval/README.md` (evaluation details and run log), `docs/LESSONS_LEARNED.md` (problems and fixes), `docs/SETUP_GUIDE.md` (setup).

---

1. Summary

When a policy document changes, someone has to read it, work out which internal procedures (SOPs) it affects, and rewrite them by hand. This prototype automates the first draft of that work: it compares an old and a new version of a public policy document, lists the changes, maps them to the SOPs they affect, drafts updated SOP text with citations, and flags anything that needs a human to check.

Where it stands: the pipeline works end to end and finds changes reliably (recall 1.0 on test cases). It over-flags unaffected SOPs and does not reliably delete requirements the policy removed. Latest repeated whole-document run: 6 of 10 test cases passed. A real-document test (MITA 2.0 to 3.0) exposed a hardware limit and a detection gap, both described below. Whether anyone needs this product is **not yet validated**.

2. Goals and non-goals

Goals

- Learn by building: a full RAG + LLM pipeline on free hardware.
- Produce a portfolio-quality, honestly evaluated project.
- Test whether "policy change to SOP update" is a workable AI task.

Non-goals

- Not a production compliance system. Output is a draft for human review, never an approved change.
- No employer, agency, member or provider data of any kind. No SSNs or PHI/PII.
- No legal or regulatory advice.

3. Constraints and data rules

Constraint	Consequence
Free compute only (Kaggle T4 x2, about 14.5 GiB per GPU)	4-bit 7B model, small prompts, section-by-section comparison for long documents
Public documents and synthetic data only	Internal SOPs are invented. Real documents are public MITA/CMS materials
Personal project, no employer ties	Disclaimer in every notebook, demo page, README and design document
Shell cannot reach medicaid.gov	Real PDFs were uploaded manually or fetched inside Kaggle

4. Architecture

```
old version + new version (public policy text)
        |
  [1 Ingest]       split into chunks, embed (MiniLM), index (FAISS)
        |
  [2 Detect]       LLM lists substantive changes (whole doc, or section by section if long)
        |
```

```
[3 Map impact]   per SOP: affected or not, structured JSON with a reason
      |
[4 Draft]        rewrite only what the change requires, cite the policy
      |
[5 Flag]         any new sentence not close to the original SOP is marked for human review
      |
[6 Report]       JSON output, redline demo page, dashboard
```

Models: Qwen2.5-7B-Instruct (4-bit, bitsandbytes) for generation; sentence-transformers all-MiniLM-L6-v2 for embeddings; FAISS for search.

Data shapes

- Document: `{id, title, text, metadata:{policy_family, version, effective_date, is_current}}`
- SOP: `{id, title, text}`
- Pipeline result: `changes_summary, impact_analysis, impact_parse_method, affected_sop_ids, drafted_updates[{sop_id, sop_title, original_text, draft_text, review_flags}]`

5. Phases, from the start to now

Phase 0 - Environment setup

Kaggle account, free T4 x2 GPU, Hugging Face token for the model download, GitHub repo. Documented in `docs/SETUP_GUIDE.md`. Free budget is about 30 GPU hours per week.

Phase 1 - Data collection (public only)

Sample public Medicaid-style policy text plus invented ("synthetic") SOPs. No real internal documents. This rule was set early and applies to everything after.

Phase 2 - Core RAG engine

Chunk text, embed with MiniLM, index with FAISS, retrieve top chunks for a question, and generate an answer with Qwen2.5-7B grounded in them, with source citations. Result: a working question-answering base. Notebook: `notebooks/compliance_ai_starter.ipynb`.

Phase 3 - Change-detection (diff) layer

- Tag each document with version metadata (family, version, effective date, current flag).
- Detect changes: send old and new text to the LLM and ask for substantive changes.
- Map changes to SOPs: the model returns structured JSON per SOP (affected yes or no, reason).
- Generate a full report. Notebook: `notebooks/phase3_diff_layer.ipynb`.

Problem found: an unaffected SOP (telehealth) was flagged because the model wrote a sentence saying it was not affected and string matching read it as a hit. **Fix:** structured JSON output with a negation-aware fallback parser (`patches/sop003_fix_cells.py`).

Phase 4 - Draft generation

For each affected SOP, draft updated text that changes only what the policy changed, with a citation. One function, `run_full_compliance_check(old_doc, new_doc, sops)`, runs everything. Notebook: `notebooks/phase4_draft_generation.ipynb`.

Problem found: the model sometimes invented sentences the policy never asked for. **Fix:** a flagger (`new_clause_flags`, similarity threshold 0.8) marks any sentence in a draft that does not closely match the original SOP, so a reviewer sees it (`patches/draft_prompt_fix_cells.py`).

First result on the sample change (prior-authorization threshold 15 to 20 visits): 2 of 3 SOPs flagged correctly, the telehealth SOP correctly left alone.

Phase 5 - Demo and export

Results exported to JSON (`data/compliance_results.json`). Two static pages: `demo/compliance_demo.html` (redline view of old vs drafted SOP text with review flags) and `demo/compliance_dashboard.html`. Both carry the disclaimer banner.

Phase 6 - GitHub packaging and documentation

Repo reorganized (notebooks, patches, demo, data, docs, eval). Swim-lane diagram of the pipeline, rewritten README, project overview PDF with product recommendations, setup guide, DISCLAIMER.md.

Phase 7 - Evaluation harness

Motivation: one hand-picked example proves little. Question: does the AI find and correctly apply real changes?

Design (in `eval/`):

- **Cases** in `eval/cases/`, each with old text, new text, SOPs (some that must not change), and an **answer key written before the AI runs**: expected changes (with key terms), affected SOP ids, required and forbidden text in edited SOPs, or "expect no change".
- **Categories:** certification (checklist and deadline changes), MITA-style, general policy (threshold change, requirement added, requirement removed), and a wording-only control.
- **Holdouts:** cases the prompts were never tuned on (`cert_02`, `policy_04`, `policy_05`) to expose overfitting.
- **Scorer** (`score.py`, no GPU): change recall, SOP precision and recall, edit correctness, drafts for unaffected SOPs, new-clause flags. Self-tested with fake pipelines (perfect, do-nothing, flag-everything) in `test_harness.py`.
- **Repeats:** each synthetic case runs 3 times because the model samples; reports always-pass, flaky and never-pass.
- **Real case:** MITA introduction (FM6), version 2.0 (2006) vs 3.0 (2012), public documents. The 2.0 to 3.0 change of interest is eight business areas becoming ten. Matching synthetic SOPs: a business-area inventory (must change) and a maturity-scoring SOP (must not be flagged).

Phase 8 - Making long documents fit (section mode)

Problem: the whole MITA pair is about 10,500 words and asked the GPU for 26 GiB. This is a design limit, not a hardware shortage.
Approach (`eval/sectioned.py`):

1. Split both documents into chunks of about 180 words.
2. Match each new chunk to its top 2 old chunks (TF-IDF cosine), non-exclusively.
3. Drop identical pairs and wording-only pairs; keep pairs whose differences involve numbers, number words or obligation words (ignoring version labels, years, months).
4. Run change detection per pair, keep a finding only if it mentions a really-changed term, drop duplicates, and cap the merged summary at about 3,500 characters.
5. Documents under about 1,500 words skip all this and are compared whole.

The real MITA pair produces 68 candidate chunk pairs, and the real 8 to 10 pair is among them (checked locally with a simulated model).

6. Key design decisions and why

Decision	Why	Trade-off
Open 4-bit 7B model on a free GPU	Zero cost, fully reproducible	Weaker than large hosted models; noisy output
Structured JSON for impact mapping	String matching misread negations	Parse failures need a fallback
Human-review flag on new clauses	Model invents text; reviewer must see it	More review work
Answer keys before running the AI	Prevents grading to fit the output	Cases take effort to write
Holdout cases and repeated runs	Avoids tuning to the test and mistaking noise for signal	More GPU time
Section-by-section for long docs	Whole docs do not fit in memory	Misses changes that span chunks; hides added/removed sentences on short docs (hence the size routing)
Draft is always "for human review"	Compliance cost of a wrong silent edit is high	No full automation claim

7. Results

Run 1 (original prompts, whole-document, 7 cases): 4 passed, 3 failed. Failures: requirements added or removed were not handled because the prompts forbade it, and SOPs that only mentioned the change indirectly were missed. Led to the v2 prompts (affected if the

change alters something the SOP states, or adds or removes a requirement for the activity the SOP performs; draft may delete a removed requirement or add one short sentence).

Run 2 (v2 prompts, 10 cases, one run each): about 4 to 5 of 10, different cases failing in different runs. The disagreement on identical input showed single runs are noisy, which led to repeated runs.

Run 3, whole-document, 3 repeats each: 6 of 10 passed. Change recall 1.0, impact precision 0.889, impact recall 1.0. Stable failures: `policy_05` removed requirement stays in the draft (0/3, a holdout, so the v2 prompt fitted only the case it was tuned on) and `control_01` false alarm on a wording-only change (0/3). `cert_01` flaky (2/3). The real MITA case ran out of memory, as expected.

Run 3, section mode, one run each: 6 of 10 passed, no crash. The capped summary fixed the out-of-memory error. But the short synthetic cases `policy_03`, `04`, `05` failed ("no draft produced") because section mode hid added and removed sentences; this led to routing short documents to whole-document comparison. On the real MITA case, 68 chunk pairs were checked and only 4 findings kept; the 8 to 10 change was missed and the maturity SOP was wrongly flagged. Whether the model or the filter is at fault is not yet known (kept findings are now printed for the next run).

8. Known weaknesses (open problems)

1. **Removed requirements stay in drafts.** Failed in all 3 whole-document runs on a holdout.
2. **False alarms.** The wording-only control fails 0/3 in whole-document mode; the maturity SOP was flagged wrongly in both real-case runs. In compliance work, noise erodes trust.
3. **Real MITA 8 to 10 change missed in section mode.** Cause not yet identified.
4. **Model noise.** Even identical inputs vary run to run.
5. **Tiny, partly self-written test set.** 10 cases. Eight are synthetic and written by me, so results say little about real-world accuracy.
6. **Untested on real SOPs or real versioned CMS documents** beyond one pair.

9. Planned fixes (next)

- **Removal handling:** ask the model only to name the sentence to delete; delete it in code; if the named text is not found verbatim in the SOP, route to human review. Deterministic and testable.
- **False alarms:** flag an SOP only when the changed terms from the detected change actually appear in it.
- **Real case:** read the printed `kept:` findings and the saved JSON to find whether the model or the filter lost the 8 to 10 change, then fix that piece only.
- **Do not change prompts again** until repeated runs show a stable failure.
- Update `eval/README.md` and this document with the new numbers after each run.

10. Roadmap

- [x] Phases 0 to 5: ingest, retrieve, detect, map, draft, flag, demo
- [x] GitHub packaging, disclaimer, swim-lane diagram, overview PDF
- [x] Evaluation harness with answer keys, holdouts, repeats, one real case
- [x] Section mode for long documents, size routing for short ones
- [ ] Fix removal and false-alarm problems, re-run with repeats
- [ ] Real MITA case passing in section mode
- [ ] 20+ cases; more real old/new public document pairs
- [ ] Customer conversations (agency staff, vendors, practitioners in other states)
- [ ] Optional: thin reporting prototype (status per SOP, synthetic quarterly summary)
- [ ] Optional: hosted backend so results persist

11. Risks

Risk	Note
Market need unvalidated	A first practitioner conversation suggested the manual process may already be good enough. Customer interviews come before more building.
Accuracy	Current numbers do not support any claim of reliability. Draft-only, human-in-the-loop is the only responsible framing.
Scope and ownership	This is a personal project built on public documents and synthetic SOPs. Check any employment agreement on side projects and IP before commercializing anything.
Public repo	The repo was made public for sharing. Set it back to private if you do not want it indexed; the link then stops working for others.

12. Repository map

```
README.md                summary, results, how to run
DISCLAIMER.md            scope and data rules
notebooks/               starter (phases 0-2), diff layer (3), draft generation (4)
patches/                 structured impact fix, draft prompt fix and flagger, prompt v2
demo/                    redline demo page, dashboard
data/                    compliance_results.json from a real run
docs/                    this document, lessons learned, setup guide, swim-lane, overview PDF
eval/                    harness: cases, scorer, section mode, mock pipelines, real case, run log
```

Independent personal project. No proprietary, client, employer or agency data, and no personal data of any kind, is used anywhere in this repository.